

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DO CEARÁ**

CAMPUS BOA VIAGEM

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RONALD VIEIRA

4º Semestre — ADS Noturno

**EVOLUÇÃO E AVALIAÇÃO DE UMA APLICAÇÃO WEB EM
LARAVEL**

Relatório técnico apresentado à disciplina de Programação Web 1 como requisito de avaliação final, contemplando a evolução, implementação, documentação, testes, versionamento e implantação de uma aplicação Web desenvolvida com PHP e Laravel.

Professor: Renato William Rodrigues de Souza

Boa Viagem — CE

2026

1 INTRODUÇÃO

O presente relatório descreve o processo de evolução de uma aplicação web desenvolvida ao longo da disciplina de Programação Web 1, utilizando o framework Laravel com a linguagem PHP. O trabalho consistiu em analisar o estado fragmentado do repositório existente, identificar suas limitações técnicas e implementar uma evolução que transformasse um conjunto de atividades isoladas em um sistema web funcional, coeso e pronto para produção.

O sistema evoluído recebeu o nome de Sistema Acadêmico Integrado e permite o gerenciamento de Alunos e Disciplinas, incluindo o relacionamento de matrícula entre eles, com operações completas de criação, leitura, atualização e exclusão (CRUD), validação de dados, testes automatizados e deploy em ambiente de produção.

2 CARACTERIZAÇÃO DA APLICAÇÃO

A aplicação é um sistema acadêmico simplificado, construído com o framework Laravel e o padrão arquitetural MVC (Model-View-Controller). Suas principais funcionalidades são o cadastro, edição, listagem e exclusão de Alunos e de Disciplinas, além da matrícula e desmatrícula de um aluno em uma ou mais disciplinas, caracterizando um relacionamento muitos-para-muitos entre as duas entidades.

As tecnologias centrais utilizadas foram PHP 8.2, o framework Laravel, o banco de dados SQLite, o motor de templates Blade, o Eloquent ORM para manipulação dos dados, Git e GitHub para versionamento, Docker para empacotamento da aplicação e o serviço Render para hospedagem em produção.

3 ANÁLISE DA SITUAÇÃO INICIAL

Ao iniciar o trabalho de evolução, o repositório Git chamado `pweb1-atividade-integrada-Ronald_Vieira` apresentava um histórico de desenvolvimento organizado em quatro branches separadas, cada uma representando uma atividade diferente da disciplina.

A branch `feature/parte-1-rotas-views` continha a implementação básica de rotas no arquivo `web.php` e views estáticas com Blade. As rotas eram simples, do tipo `Route::get()`, retornando views sem interação com banco de dados ou controllers, sem nenhuma lógica de negócio implementada.

A branch `feature/parte-2-controllers` introduziu controllers para entidades como o `DisciplinaController`, de forma básica, sem persistência de dados: os controllers retornavam

dados fixos (hardcoded) ou apenas redirecionamentos, sem comunicação real com banco de dados.

A branch *feature/parte-4-formularios* implementou formulários HTML com envio via GET e POST para uma entidade chamada Produto. Embora demonstrasse o funcionamento do CSRF token e do ciclo de requisição HTTP, os dados submetidos não eram salvos em banco de dados e não havia validação do lado do servidor.

A branch *feature/parte-5-resource* criou um Resource Controller para a entidade Aluno via `php artisan make:controller AlunoController --resource`. Todos os sete métodos gerados automaticamente (index, create, store, show, edit, update e destroy) estavam completamente vazios, contendo apenas comentários padrão do framework.

A branch *main*, que deveria representar a versão mais estável do projeto, estava no estado de uma instalação limpa do Laravel, sem nenhuma das funcionalidades desenvolvidas nas branches de atividade.

4 PROBLEMA IDENTIFICADO

A análise do repositório revelou um conjunto de problemas técnicos que impediam que o projeto fosse considerado uma aplicação web funcional:

- Ausência de persistência de dados: nenhuma das branches possuía migrations para entidades de negócio, além das tabelas padrão do Laravel (users, cache e jobs);
- Falta de Models e ORM: sem models definidos, não era possível utilizar o Eloquent para interagir com o banco de forma orientada a objetos;
- Controllers sem implementação: o Resource Controller de Alunos era apenas um esqueleto gerado automaticamente, sem nenhuma linha de código funcional;
- Ausência de validação de dados: nenhum formulário validava os dados do lado do servidor;
- Sem testes automatizados: não havia testes para verificar rotas, banco de dados ou regras de negócio;
- Fragmentação e falta de integração: as partes existiam isoladas em branches nunca integradas entre si;
- Interface sem identidade visual: páginas HTML básicas, sem layout compartilhado e sem mensagens de feedback ao usuário.

Diante desse cenário, a evolução necessária não era apenas a adição de funcionalidades pontuais, mas a reconstrução integrada da aplicação sobre a fundação do Laravel, unificando os conceitos abordados nas atividades em um sistema real e funcional.

5 PROPOSTA DE EVOLUÇÃO

A solução planejada consistiu em consolidar as quatro branches fragmentadas em uma aplicação única, implementada diretamente na branch main, cobrindo as entidades Aluno e Disciplina com seu relacionamento de matrícula. Foram planejados: a criação de migrations para as tabelas de negócio; a definição dos models com relacionamento Eloquent muitos-para-muitos; a criação de uma camada de serviço dedicada às regras de matrícula; a reescrita dos controllers com validação de dados; a criação de views Blade com layout compartilhado; a implementação de testes automatizados; e, por fim, a publicação da aplicação em um serviço de hospedagem acessível pela internet.

Os resultados esperados eram um sistema íntegro, testável, versionado de forma incremental e disponível publicamente, aplicando na prática os conceitos de rotas, Models, Migrations, ORM Eloquent, relacionamentos, Services, validação de dados, banco de dados, testes e deploy estudados na disciplina.

6 IMPLEMENTAÇÃO

A implementação foi realizada inteiramente na branch main, que passou a representar a versão integrada e evoluída do projeto. As decisões técnicas foram guiadas pelos princípios do framework Laravel e pelas boas práticas de desenvolvimento web backend.

6.1 Banco de Dados e Migrations

Foram criadas três migrations. A primeira, `create_alunos_table`, cria a tabela alunos com os campos `id`, `nome`, `email` (único), `matricula` (única), `data_nascimento` e os timestamps padrão do Laravel. A segunda, `create_disciplinas_table`, cria a tabela disciplinas com `id`, `nome`, `codigo` (único) e `carga_horaria`. A terceira, `create_aluno_disciplina_table`, cria a tabela pivô do relacionamento muitos-para-muitos, com chaves estrangeiras `aluno_id` e `disciplina_id`, ambas com deleção em cascata.

A decisão de utilizar migrations se justifica pelo fato de que elas permitem o versionamento do esquema do banco de dados: em vez de criar tabelas manualmente, o banco pode ser recriado em qualquer ambiente com um único comando, o `php artisan migrate`, o que é essencial tanto para o desenvolvimento local quanto para o deploy em produção.

Os campos `email` e `matricula` foram marcados como únicos porque são identificadores de cada aluno e não devem se repetir; o mesmo critério se aplica ao campo `codigo` de cada disciplina. A tabela pivô `aluno_disciplina` foi criada porque o

relacionamento entre alunos e disciplinas é do tipo N:M — um aluno pode estar matriculado em várias disciplinas e uma disciplina pode ter vários alunos —, o que exige uma tabela intermediária pelo modelo relacional de banco de dados.

O banco de dados SQLite foi escolhido por sua simplicidade operacional: não requer instalação de um servidor de banco separado, funciona com um único arquivo e é totalmente suportado pelo Laravel, sendo a escolha mais prática tanto para o projeto acadêmico quanto para o deploy gratuito no Render.

6.2 Models e Eloquent ORM

Foram criados os models `Aluno` e `Disciplina`. O model `Aluno` define o atributo `fillable` com os campos `nome`, `email`, `matricula` e `data_nascimento`, e possui o método `disciplinas`, que retorna um relacionamento `belongsToMany` com o model `Disciplina`, especificando a tabela pivô `aluno_disciplina`. O model `Disciplina` segue a mesma estrutura, com o método inverso `alunos`.

O atributo `fillable` define quais campos podem ser preenchidos via atribuição em massa (`mass assignment`), o que é uma medida de segurança: sem ele, um atacante poderia enviar campos extras em uma requisição HTTP para sobrescrever dados que não deveriam ser alterados.

O método `belongsToMany` implementa o relacionamento N:M via Eloquent. Ao defini-lo nos dois models, o framework passa a oferecer métodos de alto nível como `attach`, `detach` e `sync` para gerenciar a tabela pivô sem a necessidade de escrever SQL manualmente. Por exemplo, para matricular um aluno em uma disciplina, basta chamar o método `attach` na relação `disciplinas` do objeto `aluno`; o Eloquent se encarrega de inserir o registro correto na tabela pivô.

6.3 Camada de Serviço (Service Layer)

Foi criada a classe `app/Services/MatriculaService.php`, responsável pelas regras de negócio relacionadas às matrículas. O método `matricular` recebe um objeto `Aluno` e um identificador de disciplina, verifica se a disciplina existe (`findOrFail`), verifica se o aluno já está matriculado consultando a relação no banco e, caso não esteja, realiza o `attach`. O método `desmatricular` executa o `detach` na relação.

A separação de responsabilidades segue o princípio da responsabilidade única: sem a Service Layer, toda a lógica de matrícula ficaria dentro do `AlunoController`, dificultando a manutenção e os testes. Ao isolar a regra de negócio em uma classe dedicada, ela pode ser testada de forma independente e reutilizada em outros pontos do sistema sem duplicação de código. O `MatriculaService` é injetado no `AlunoController` via construtor, seguindo a prática

de injeção de dependência recomendada pelo Laravel.

6.4 Controllers e Validação

Foram implementados dois controllers com CRUD completo: o `AlunoController`, com os métodos `index`, `create`, `store`, `show`, `edit`, `update`, `destroy`, `matricular` e `desmatricular`; e o `DisciplinaController`, com `index`, `create`, `store`, `edit`, `update` e `destroy`.

A validação foi implementada com o método `validate` do objeto `request`. No método `store` do `AlunoController`, por exemplo, as regras exigem que o nome seja obrigatório e tenha no máximo 255 caracteres; que o email seja obrigatório, válido e único na tabela `alunos`; que a matrícula seja obrigatória e única; e que a `data_nascimento` seja obrigatória e válida. No método `update`, a regra `unique` foi adaptada para ignorar o próprio registro sendo editado, permitindo que o e-mail já exista no banco desde que pertença ao próprio aluno.

O uso do `Route Model Binding`, com parâmetros tipados como `Aluno` e `Disciplina` nos métodos do controller, elimina a necessidade de buscar o registro manualmente: o Laravel faz a busca automaticamente e retorna um erro 404 se o registro não existir.

6.5 Rotas

O arquivo `routes/web.php` foi atualizado com `Route::resource` para `alunos` e `disciplinas`, gerando automaticamente as sete rotas convencionais de CRUD para cada entidade. Foram adicionadas ainda duas rotas manuais: uma rota `POST` para `alunos/{aluno}/matricular`, nomeada `alunos.matricular`, e uma rota `DELETE` para `alunos/{aluno}/desmatricular/{disciplina}`, nomeada `alunos.desmatricular`. O método `HTTP DELETE` foi escolhido para a desmatrícula por representar semanticamente a remoção de um recurso, seguindo os princípios `REST`.

6.6 Views e Interface

Foram criadas as views `layouts/app.blade.php` (layout base com navbar e sistema de flash messages), `alunos/index`, `create`, `edit` e `show` (a última funcionando também como interface de matrícula e desmatrícula), `disciplinas/index`, `create` e `edit`, além do arquivo `public/css/style.css` com a folha de estilos própria da aplicação.

O layout compartilhado, com as diretivas `@extends` e `@yield` do Blade, evita a repetição do HTML da navbar e do head em cada view. O sistema de flash messages (`session('success')` e `session('error')`) fornece feedback imediato ao usuário após cada operação. O uso de CSS próprio, sem frameworks externos, permite controle total sobre o visual sem dependências desnecessárias.

7 APLICAÇÃO DOS CONCEITOS DE PROGRAMAÇÃO WEB 1

Além de Laravel e Git, exigidos como base do trabalho, foram aplicados os seguintes conceitos técnicos estudados na disciplina:

- **Rotas:** uso de Route::resource e rotas customizadas seguindo o padrão REST;
- **Models e ORM Eloquent:** Aluno e Disciplina com relacionamento belongsToMany;
- **Migrations:** versionamento do esquema do banco com três migrations dedicadas às entidades de negócio;
- **Relacionamentos entre modelos:** relacionamento muitos-para-muitos via tabela pivô aluno_disciplina;
- **Services:** camada de serviço MatriculaService isolando as regras de negócio de matrícula;
- **Validação de dados:** uso do método validate nos controllers, com regras de obrigatoriedade e unicidade;
- **Testes:** testes de Feature automatizados cobrindo criação de aluno e matrícula;
- **Versionamento com Git:** histórico de commits organizados por funcionalidade;
- **Containerização e deploy:** Dockerfile customizado e publicação em serviço de hospedagem.

8 BANCO DE DADOS, MODELS, ELOQUENT E MIGRATIONS

O banco de dados utilizado é o SQLite, estruturado em três tabelas: alunos, disciplinas e a tabela pivô aluno_disciplina, responsável pelo relacionamento muitos-para-muitos entre as duas primeiras. As migrations garantem que essa estrutura possa ser recriada de forma idêntica em qualquer ambiente, bastando executar php artisan migrate.

Os models Aluno e Disciplina utilizam o Eloquent ORM para abstrair o acesso ao banco, eliminando a necessidade de escrever SQL manualmente para as operações de CRUD e de gerenciamento do relacionamento N:M (attach e detach), conforme detalhado na seção de implementação.

9 ROTAS, CONTROLLERS, VIEWS E SERVICES

A camada de rotas expõe os endpoints RESTful de Alunos e Disciplinas, além dos endpoints específicos de matrícula e desmatrícula. Os controllers concentram a validação de entrada e delegam as regras de negócio de matrícula à camada de serviço MatriculaService, mantendo uma separação clara de responsabilidades. As views Blade consomem os dados enviados pelos controllers e compartilham um layout único, garantindo consistência visual

e de navegação em toda a aplicação.

10 TESTES E EVIDÊNCIAS

Foram implementados testes de Feature no arquivo `tests/Feature/AlunoTest.php`, utilizando o trait `RefreshDatabase` para garantir que cada teste execute com um banco de dados limpo.

O teste `test_pode_criar_aluno` simula uma requisição HTTP POST para o endpoint `/alunos` com dados válidos, verifica se a resposta é um redirecionamento para `/alunos` e se o registro foi efetivamente persistido no banco, usando o método `assertDatabaseHas`.

O teste `test_pode_matricular_aluno_em_disciplina` cria um aluno e uma disciplina diretamente no banco, simula uma requisição POST para o endpoint de matrícula e verifica se o registro foi inserido na tabela pivô `aluno_disciplina`, validando o funcionamento integrado do `MatriculaService`, do controller e da rota.

O resultado da execução via `php artisan test` foi de testes passando sem falhas, com todas as asserções verificadas com sucesso, confirmando o funcionamento correto do CRUD de alunos e do processo de matrícula.

11 VERSIONAMENTO COM GIT

O projeto utilizou Git desde o início da disciplina, evoluindo a partir das branches de atividades anteriores (`feature/parte-1-rotas-views`, `feature/parte-2-controllers`, `feature/parte-4-formularios` e `feature/parte-5-resource`) até a consolidação de todo o sistema na branch `main`. Entre os principais marcos do histórico de commits estão a estrutura inicial do projeto Laravel, a criação do README com informações do projeto, a evolução do projeto com a adição de Aluno, Disciplina e seu relacionamento, a adição do Dockerfile para deploy, e os ajustes de configuração de HTTPS realizados em produção.

Durante o processo de integração do trabalho, foi necessário utilizar o comando `git pull` com a opção `--allow-unrelated-histories` para mesclar o histórico local com o histórico já existente no repositório remoto, situação típica de um projeto de evolução que reaproveita um repositório criado anteriormente.

12 IMPLANTAÇÃO E DEPLOY

A aplicação foi implantada no serviço Render (render.com), no plano gratuito, utilizando a estratégia de deploy via Docker. O Render não possui runtime nativo para PHP, o que exigiu a criação de um Dockerfile customizado baseado na imagem oficial

php:8.2-cli.

O Dockerfile configura a instalação das extensões PHP necessárias (pdo, pdo_sqlite e zip), a instalação do Composer, a criação do arquivo do banco SQLite e a configuração de permissões dos diretórios storage, bootstrap/cache e database, além do comando de inicialização que executa o cache de configurações, as migrations e o servidor web na porta definida pela variável de ambiente PORT.

As variáveis de ambiente configuradas no Render foram: APP_KEY, com a chave de segurança da aplicação; APP_ENV, com o valor production; APP_DEBUG, com o valor false, para desativar a exibição de erros detalhados; DB_CONNECTION, com o valor sqlite; e APP_URL, com o endereço público da aplicação.

O endereço da aplicação publicada é <https://pweb1-atividade-integrada-ronald-vieira.onrender.com>.

Aviso sobre o plano gratuito: por se tratar de um plano gratuito, o serviço entra em modo de espera após 15 minutos sem uso, fazendo com que o primeiro acesso após um período de inatividade leve cerca de 30 segundos para carregar, enquanto a aplicação é reativada. Após esse primeiro carregamento, a navegação volta ao normal. Esse comportamento é esperado em hospedagens gratuitas e não representa um erro da aplicação.

Durante o processo de implantação, foram enfrentadas três dificuldades principais, descritas a seguir.

A primeira foi a criação manual do arquivo do banco SQLite: ao rodar a aplicação localmente pela primeira vez, o sistema acusou que o arquivo de banco de dados não existia no caminho configurado, pois o Laravel, ao usar SQLite, não cria esse arquivo automaticamente. A solução foi criar o arquivo manualmente antes de executar o php artisan migrate, prática posteriormente incorporada ao próprio Dockerfile de produção.

A segunda foi um problema de repositório Git aninhado: durante a configuração do deploy, identificou-se que a pasta do projeto Laravel estava sendo tratada como um submódulo, pois havia um diretório .git tanto na pasta raiz quanto na subpasta com o código-fonte. Isso fazia com que o serviço de hospedagem tentasse clonar o repositório externo em vez do repositório interno com o código da aplicação. A solução envolveu identificar a estrutura correta de pastas e trabalhar diretamente a partir do repositório interno, já conectado ao remoto correto.

A terceira foi um problema de conteúdo misto entre HTTP e HTTPS em produção: após o primeiro deploy bem-sucedido, o layout da aplicação não carregava corretamente,

pois o CSS não era aplicado. A investigação, feita pela inspeção do código-fonte da página, revelou que o Laravel estava gerando URLs de assets com o protocolo HTTP, enquanto a página era servida em HTTPS; o navegador bloqueou o carregamento do CSS por política de segurança de conteúdo misto. A causa raiz é que o proxy reverso do Render repassa as requisições ao container internamente via HTTP, impedindo que o Laravel detectasse o HTTPS da conexão original do usuário. A solução foi adicionar a instrução `URL::forceScheme('https')` no método `boot()` da classe `AppServiceProvider`, condicionada ao ambiente de produção, forçando o Laravel a sempre gerar URLs com HTTPS quando hospedado.

13 COMPARAÇÃO ANTES/DEPOIS

Situação anterior: quatro branches isoladas, sem integração entre si, sem persistência de dados, sem Models, sem validação, sem testes e sem nenhuma versão publicada na internet. O Resource Controller de Alunos era apenas um esqueleto vazio.

Situação após a evolução: uma aplicação única e integrada, com CRUD completo de Alunos e Disciplinas, relacionamento de matrícula funcional via Eloquent, camada de serviço dedicada às regras de negócio, validação de dados em todos os formulários, testes automatizados cobrindo os principais fluxos, interface com layout compartilhado e identidade visual própria, e publicação em ambiente de produção acessível publicamente.

Benefícios obtidos: o sistema passou a ser efetivamente utilizável, com garantia de integridade dos dados por meio das validações e das restrições de unicidade no banco, além de maior confiabilidade proporcionada pelos testes automatizados e maior facilidade de manutenção graças à separação de responsabilidades entre controllers e camada de serviço.

Limitações que permaneceram: o projeto não implementa autenticação de usuários nem controle de acesso, e a hospedagem gratuita utilizada apresenta o comportamento de spin down já descrito na seção de implantação e deploy.

14 RESULTADOS E DISCUSSÃO

O resultado final é uma aplicação funcional que integra, em um único sistema, conceitos que antes estavam dispersos e incompletos em quatro branches diferentes. A adoção da camada de serviço para as regras de matrícula demonstrou, na prática, o valor da separação de responsabilidades: a lógica de negócio pôde ser testada de forma isolada, sem depender diretamente da camada HTTP dos controllers.

O processo de deploy revelou desafios que vão além do código da aplicação em si, como a diferença entre o comportamento de um ambiente local e o de um ambiente de produção hospedado atrás de um proxy reverso (o caso do conteúdo misto HTTP/HTTPS). A resolução desses problemas exigiu não apenas conhecimento de Laravel, mas também compreensão de como aplicações web se comportam quando implantadas na internet, o que agregou valor prático ao aprendizado da disciplina.

15 CONSIDERAÇÕES FINAIS

A evolução do sistema demonstrou na prática a integração dos conceitos estudados ao longo da disciplina. Partindo de um conjunto de atividades isoladas sem persistência de dados, o projeto foi transformado em uma aplicação web funcional que implementa princípios fundamentais do desenvolvimento backend com Laravel: rotas RESTful, ORM com relacionamentos, camada de serviço, validação de dados, testes automatizados e deploy em produção.

Entre os conceitos efetivamente aplicados e compreendidos ao longo do desenvolvimento, destacam-se: (1) migrations e versionamento de esquema de banco de dados; (2) relacionamentos Eloquent do tipo muitos-para-muitos; (3) camada de serviço (Service Layer) para separação de regras de negócio; (4) validação de dados no lado do servidor; e (5) versionamento com Git aplicado à integração de históricos divergentes.

As dificuldades encontradas, desde a configuração do banco SQLite até o diagnóstico do problema de conteúdo misto em produção, reforçaram a importância de compreender não apenas o código, mas o ambiente em que ele opera. Cada problema identificado gerou uma solução incorporada de forma permanente ao projeto, tornando o sistema mais robusto. O resultado final é uma aplicação que segue as convenções do framework, aplica padrões arquiteturais reconhecidos e está hospedada em ambiente de produção acessível publicamente.

16 REFERÊNCIAS

LARAVEL. Laravel Documentation. Disponível em: <https://laravel.com/docs>. Acesso em: set. 2026.

OTWELL, Taylor. Laravel: The PHP Framework for Web Artisans. Disponível em: <https://laravel.com>. Acesso em: set. 2026.

RENDER. Render Documentation — Docker Deploy. Disponível em: <https://render.com/docs/docker>. Acesso em: set. 2026.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

MARTIN, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston: Prentice Hall, 2017.

MOZILLA DEVELOPER NETWORK. Mixed Content. Disponível em: https://developer.mozilla.org/en-US/docs/Web/Security/Mixed_content. Acesso em: set. 2026.

17 ANEXOS

Anexo A — Histórico de commits

A tabela a seguir apresenta os principais commits realizados durante a evolução e o deploy do projeto, extraídos do histórico do repositório (git log).

Commit	Mensagem
1e4b22f	chore: estrutura inicial do projeto Laravel
0ad3e98	Create README with project information
bbe9c2f	Evolução do projeto: adiciona Aluno, Disciplina e relacionamento
eace6a3	adiciona Dockerfile para deploy no Render
43e05f8	adiciona build de assets Vite no Dockerfile
9e16501	força HTTPS nas URLs geradas em producao
27c61fb	corrige encoding do AppServiceProvider (remove BOM)
20e4ce5	corrige README: remove arquivo antigo, adiciona README.md atualizado
e92c0fb	adiciona aviso sobre spin down do plano gratuito no Render

Anexo B — Principais comandos utilizados

Abaixo estão listados os principais comandos de terminal utilizados durante o versionamento, a integração e o deploy do projeto.

Versionamento e integração com o repositório remoto

```
git init
git add .
git commit -m "mensagem do commit"
git branch -M main
git remote add origin
https://github.com/Ronald-br/pweb1-atividade-integrada-Ronald_Vieira.git
git pull origin main --allow-unrelated-histories
git push -u origin main
```

Geração da chave de aplicação do Laravel

```
php artisan key:generate
```

Build e deploy via Docker (executado pelo Render)

```
composer install --optimize-autoloader --no-dev  
npm install && npm run build  
php artisan config:cache  
php artisan migrate --force  
php artisan serve --host=0.0.0.0 --port=10000
```

Execução dos testes automatizados

```
php artisan test
```

As evidências completas de código, estrutura de diretórios, testes e histórico integral de commits podem ser verificadas diretamente no repositório do projeto, disponível em: https://github.com/Ronald-br/pweb1-atividade-integrada-Ronald_Vieira.